Distributed Systems (Labs 2–5 & Midterm Defense)

Replicated Counter Service

Ilya Lishkevich

1. Requirements under test

Our service is a distributed counter using gRPC in Python. It supports increment and get operations with majority quorum (2 of 3) and Lamport clocks.

Task	Requirement	Verified By
A2	At-most-once RPC, retries, idempotency	test_increment_applies_delta, test_duplicate_key_not_reapplied, test_retry_after_timeout_is_safe
A3	Thread safety under concurrent clients	test_concurrent_increments_exact
B1	Lamport logical clocks on messages	Captured trace log (logs/trace.log)
C1	3 replicas with 2-of-3 quorum commit	test_majority_commit_two_acks, test_no_commit_below_majority

RPC Semantics: We provide *at-most-once* semantics. The client makes a unique idempotency_key (UUID) for each action and reuses it on retries. The server saves finished keys in _seen under a threading.Lock(). If a retry arrives, the server returns the cached result with was_duplicate = true and does not add the delta again.

Task C1 Discussion:

- *Partial write anomaly:* If only 1 replica gets a write and 2 are down, the write fails for the client (1 < 2), but that single replica already updated its counter in memory. Raft prevents this by writing to a log first and applying only after quorum confirms.
- *Get consistency risk:* Reading from a single replica can return old data (stale read) if that replica missed the last update.

2. Test environment

- **OS:** Windows 11 (64-bit)
- **Python:** 3.14.5
- **Packages:** grpcio (1.84.0), grpcio-tools (1.84.0), protobuf (7.36.2), pytest (9.1.1)
- **Machine:** Standard multi-core x86_64 PC, 16 GB RAM

3. Test-case table

All 11 tests use the Arrange-Act-Assert pattern in pytest.

ID	Purpose	Expected result	Actual result	Pass/ Fail
1	Single increment	Value increases by delta (+5)	Value is 5	**PASS**
2	Idempotency check	Same key returns duplicate=True	Value stays 5	**PASS**
3	Concurrency (Task A3)	2 clients × 1,000 adds = 2,000	Value is 2000	**PASS**
4	Missing key	Returns found=False with out crash	found=False	**PASS**
5	Retry after timeout	Counter moves only once	Value is 10	**PASS**
6	Quorum with 1 dead node	Write succeeds with 2/3 acks	acked=2	**PASS**
7	Below majority (2 dead)	Write fails (1 ack is not enough)	success=False	**PASS**
8	State convergence	All 3 replicas hold the same value	All have 10	**PASS**
9	Crash during request	Surviving 2 replicas commit write	success=True	**PASS**
10	Duplicate network call	Second call returns duplicate=True	Counter static	**PASS**
11	Node latency > 2.0s	Fast nodes reach quorum (2 acks)	success=True	**PASS**

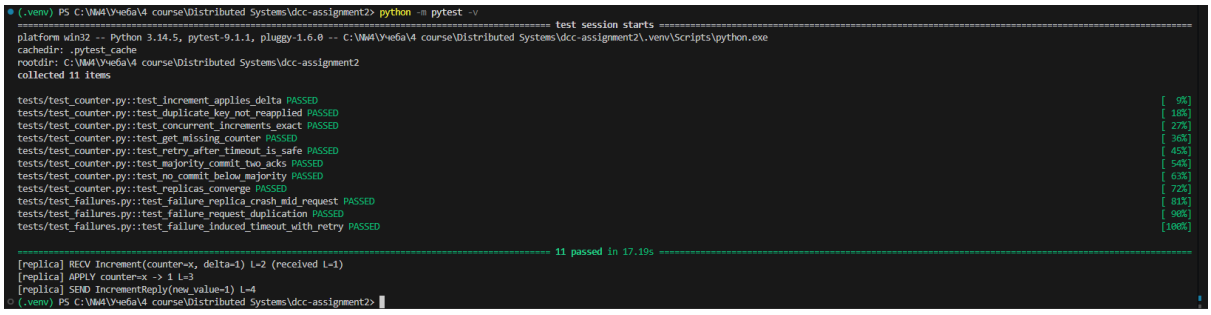


Figure 1. Automated test suite output (11/11 passed).

4. Failure-injection results

1. **Replica crash mid-request:** We killed Replica 2 while a request was running (servers[2].stop(0)). The client received answers from Replicas 0 and 1, reached quorum (2 >= 2), and committed without throwing an exception.
2. **Request duplication:** We sent the same key same-key-42 twice. The first call applied +5. The second returned was_duplicate=True and kept the value at 5.
3. **Induced timeout:** Replica 2 was given a 2500 ms delay (timeout is 2.0s). The client did not block on it; Replicas 0 and 1 answered immediately, so quorum (2/3) succeeded.

5. Performance table and interpretation (Task C4)

We ran 2,000 requests per configuration using perf_benchmark.py. Percentile p95 was computed as [0.95*n].

Configuration	Median latency (ms)	p95 latency (ms)	Requests
Single replica, 1 client	1.64	2.31	2000
Single replica, 16 clients	20.74	27.58	2000
Quorum (3 replicas), 1 client	3.73	4.50	2000
Quorum (3 replicas), 16 clients	61.84	85.37	2000

Figure 2. Performance benchmark table.

Interpretation:
The quorum path is slower because the client sends requests to three servers over the network and must wait for the second-fastest reply. With 16 clients, latency increases to 61.84 ms (median) and 85.37 ms (p95) due to lock contention on threading.Lock() inside the server. This latency is completely acceptable for a social network like counter, because increments run in the background, users do not notice ~85 ms, and the service survives server crashes.

6. Known limitations

- Data lost on restart (In-Memory only): Counters and deduplication keys are kept in Python dictionaries (_values, _seen). If servers restart, all data is lost. A production system must save writes to a Write-Ahead Log (WAL) on disk.
- Stale reads on Get: Get queries only one replica. If that replica missed the last update, it returns old data. To ensure consistency, reads should also query a quorum (R >= 2).

7. Task B2: Ordering analysis

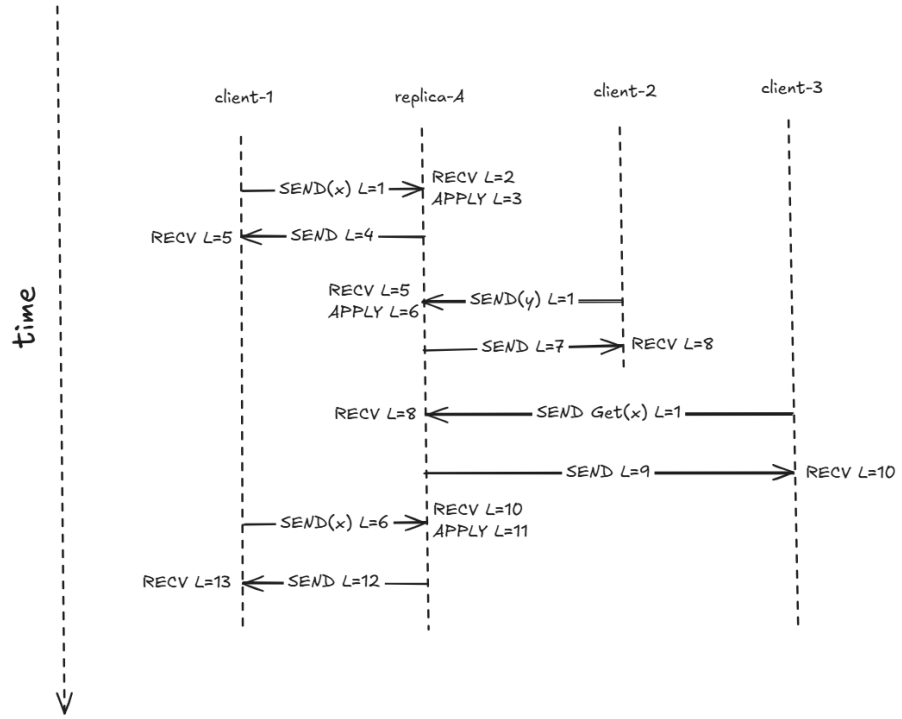


Figure 3. Happened-before causal diagram.

Two causally ordered pairs:

- Pair 1 (Message edge): [client-1] SEND(x) L=1 -> [replica-A] RECV(x) L=2. These events are connected by a message sent over the network. In Lamport clocks, sending a message always happens before receiving it (L=1 < L=2).
- Pair 2 (Causal chain): [replica-A] APPLY L=3 -> [client-1] SEND(x) L=6. There is a clear chain of events: APPLY (L=3) happens locally on the server before SEND Reply (L=4), which travels over the network to [client-1] RECV (L=5), which then happens locally before [client-1] sends its second request SEND (L=6).

One pair of concurrent events:

- [client-1] SEND(x) L=1 and [client-2] SEND(y) L=1 are concurrent. Both have L=1 and started on different machines without communicating with each other. A smaller or equal Lamport value does not mean one event happened before the other in real physical time.

Limitation of Lamport clocks:

- Lamport clocks compress time into a single number. Because of this, if L(a) < L(b), you cannot tell if event a caused event b or if they are concurrent. Vector clocks solve this problem by storing a clock for each process, which allows detecting concurrency directly.

Reflection questions

Q1. After a timeout, why is it impossible to tell request lost from reply lost, and how does idempotency fix this?

A timeout is just a 2-second timer that ended on the client. The client cannot see what happened on the network. It does not know if the request was dropped before reaching the server, or if the server updated the counter and only the response was dropped on the way back.

Our idempotency key fixes this problem. The client keeps the same UUID on retries. If the request was lost, the server executes it normally. If the reply was lost, the server sees this UUID in self._seen, does not increment the counter a second time, and simply returns the saved result.

Q2. Why a smaller Lamport value does not mean the event happened first:

Lamport clocks only increase when a node does a local action or gets a message. If Client 1 did several actions, its clock went up to L=6. If Client 2 was idle and just started, its clock was L=1. Even though 1 < 6, it does not mean Client 2 started earlier in real physical time. Because Client 1 and Client 2 never sent messages to each other, they are simply concurrent events.

Q3. Which failures can the service tolerate, and Safety vs Liveness (Week 5):

With 3 replicas and a majority quorum of 2:

- 1 failure: The system tolerates 1 dead replica because 2 replicas are still alive to confirm the write.
- 2 failures: If 2 replicas crash, we only have 1 replica left. The write fails because 1 is less than the majority of 2.

Relating to Week 5: "Safety" means no wrong or conflicting data is committed, and "Liveness" means the system keeps answering requests. When 2 replicas die, the system chooses Safety over Liveness: it returns an error and refuses to write, rather than letting 1 replica commit inconsistent data.

Q4. Smallest change to survive restarts, and Week 6 mechanism:

Right now, our server keeps everything in memory (Python dictionaries), so restarting a server deletes all counters. The smallest change is to write each increment and idempotency key into a simple log file on disk before replying to the client. When the server restarts, it reads this file to restore the dictionary.

This anticipates the log replication mechanism in Raft (mentioned on page 2 of the assignment for Week 6), where nodes use an append-only log on disk to keep and recover state.

Appendices

```
(.venv) PS C:\NW4\Учеба\4 course\Distributed Systems\dcc-assignment2> pip freeze
colorama==0.4.6
grpcio==1.84.0
grpcio-tools==1.84.0
iniconfig==2.3.0
packaging==26.3
pluggy==1.6.0
protobuf==7.36.2
Pygments==2.21.0
pytest==9.1.1
setuptools==84.0.0
typing_extensions==4.16.0
```

Appendix A: Environment Package Manifest (pip freeze)

logs > trace.log

```
1   [replica-A] RECV Increment(counter=x, delta=1) L=2 (received L=1)
2   [replica-A] APPLY counter=x -> 1 L=3
3   [replica-A] SEND IncrementReply(new_value=1) L=4
4   [replica-A] RECV Increment(counter=y, delta=1) L=5 (received L=1)
5   [replica-A] APPLY counter=y -> 1 L=6
6   [replica-A] SEND IncrementReply(new_value=1) L=7
7   [replica-A] RECV Get(counter=x) L=8 (received L=1)
8   [replica-A] SEND GetReply(value=1, found=True) L=9
9   [replica-A] RECV Increment(counter=x, delta=1) L=10 (received L=6)
10  [replica-A] APPLY counter=x -> 2 L=11
11  [replica-A] SEND IncrementReply(new_value=2) L=12
12  [replica-A] RECV Increment(counter=y, delta=1) L=13 (received L=9)
13  [replica-A] APPLY counter=y -> 2 L=14
14  [replica-A] SEND IncrementReply(new_value=2) L=15
15  [replica-A] RECV Get(counter=y) L=16 (received L=11)
16  [replica-A] SEND GetReply(value=2, found=True) L=17
17
18  [client-1] SEND Increment(counter=x, delta=1) L=1
19  [client-2] SEND Increment(counter=y, delta=1) L=1
20  [client-1] RECV IncrementReply(new_value=1) L=5 (received L=4)
21  [client-2] RECV IncrementReply(new_value=1) L=8 (received L=7)
22  [client-3] SEND Get(counter=x) L=1
23  [client-3] RECV GetReply(value=1) L=10 (received L=9)
24  [client-1] SEND Increment(counter=x, delta=1) L=6
25  [client-2] SEND Increment(counter=y, delta=1) L=9
26  [client-1] RECV IncrementReply(new_value=2) L=13 (received L=12)
27  [client-2] RECV IncrementReply(new_value=2) L=16 (received L=15)
28  [client-3] SEND Get(counter=y) L=11
29  [client-3] RECV GetReply(value=2) L=18 (received L=17)
```

Appendix B: Captured Event Log Trace (logs/trace.log)